Aula 32: Gerenciamento de Estado Global

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- ESTADO LOCAL VS. ESTADO GLOBAL
- PROP DRILLING
- ARQUITETURA FLUX
- CONTEXT API
- INTRODUÇÃO AO REDUX
- MELHORES PRÁTICAS
- EXERCÍCIOS

Introdução

COMPONENTIZAÇÃO

- Uma das bases do desenvolvimento de uma aplicação React e um dos principais atrativos da biblioteca
- Permite que diferentes funcionalidades sejam separadas em componentes, que podem, ou não, ter uma relação entre si
- O que facilita o reuso e permite que cada um deles controle e tenha acesso às suas próprias informações

Introdução

ESTADOS

- Considerar uma aplicação em que o usuário possa escolher o tema de um site, por exemplo
- É necessário que diversos componentes da aplicação saibam qual tema foi escolhido
- Provavelmente, essa informação será definida como um *state* dentro do componente que permite a escolha do tema

Introdução

ESTADOS

- Em React, estado representa dados que mudam ao longo do tempo
- Esses dados afetam a renderização da interface (UI)
- São considerados a "memória" dos componentes
- Um exemplo é escolher o tema de um site
- useState, por exemplo lida bem em escopo local

Introdução

ESTADOS

- Exemplos:
  - O valor de um campo de formulário
  - Se um menu suspenso está aberto ou fechado
  - A lista de itens em um carrinho de compras
  - As informações do usuário logado

Introdução

ESTADOS

- No exemplo do tema di site, o desafio surge porque outros componentes da aplicação não têm acesso direto a esse state para que definam seu tema
- Portanto, é preciso que essa informação seja compartilhada de maneira global dentro do projeto
- O problema aqui é que estado local não escala para dados compartilhados de maneira global

Estado Local vs. Estado Global

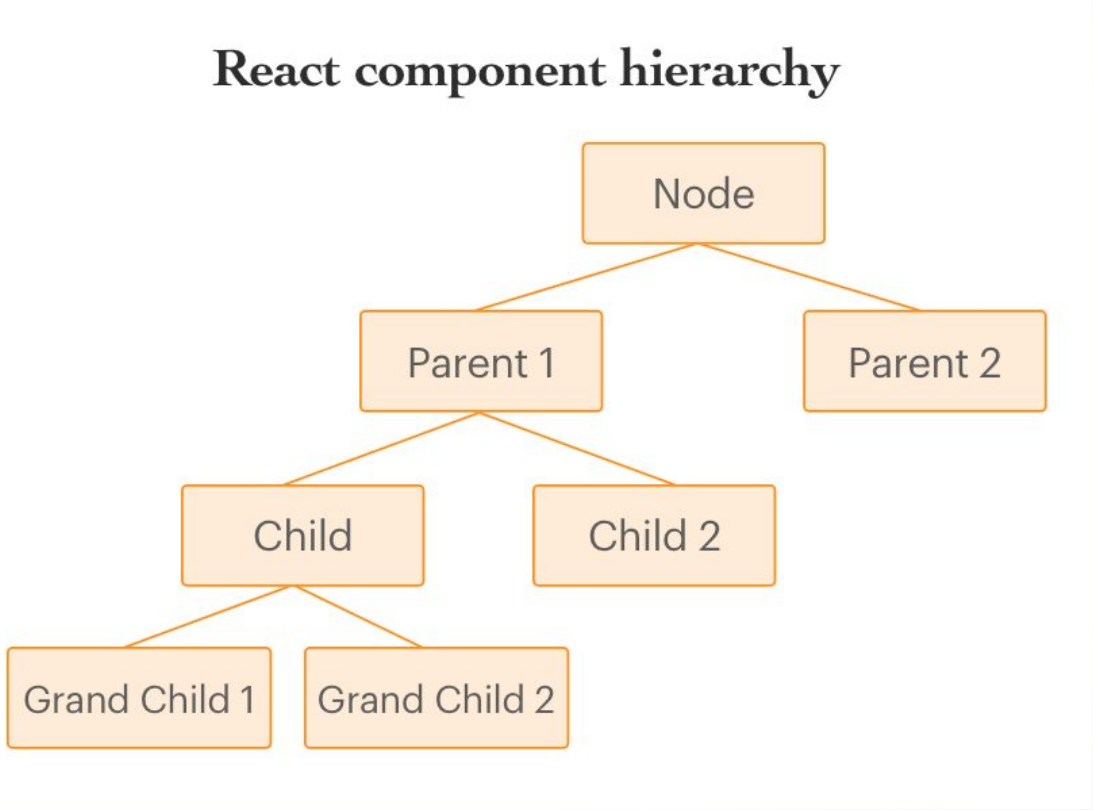
- Estado Local:
  - Pertence a um único componente.
  - Criado e gerenciado com useState ou useReducer.
  - Exemplo: O estado de "visibilidade" de uma senha em um campo de input

Estado Local vs. Estado Global

- Estado Global:
  - Necessário em múltiplos componentes, em diferentes partes da aplicação.
  - Exemplos: Informações do usuário, tema da aplicação (claro/escuro), itens do carrinho

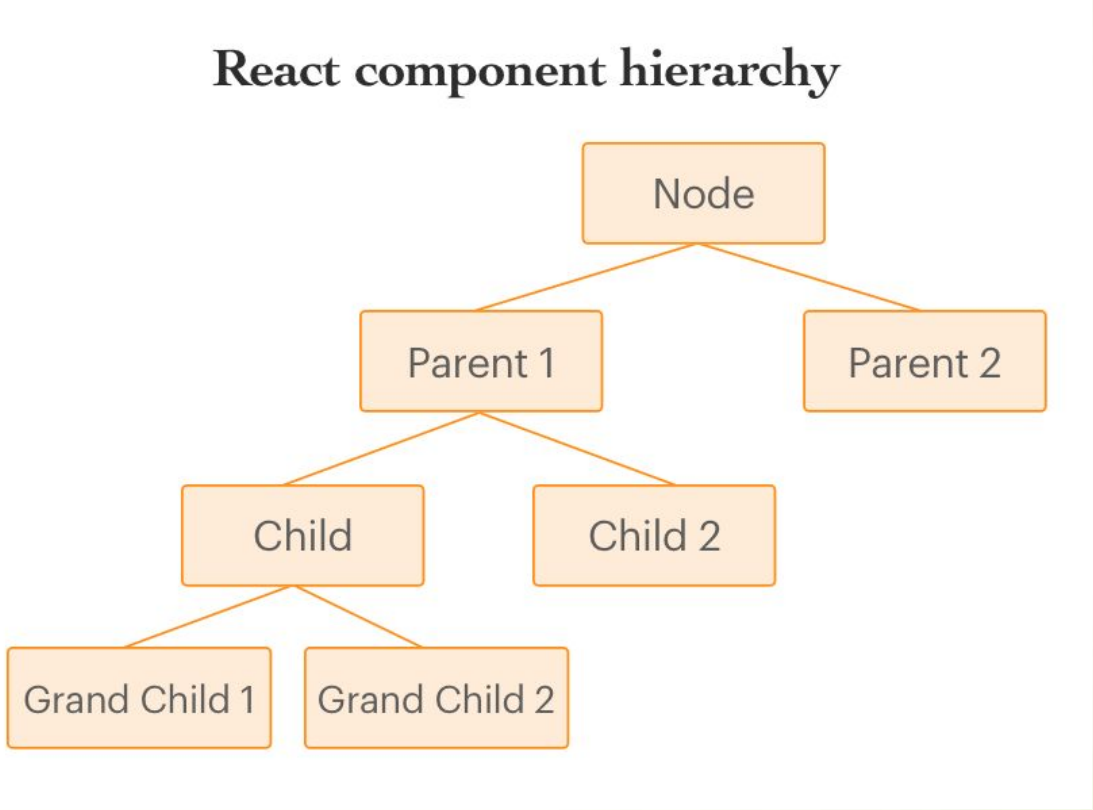
Estado Local vs. Estado Global

- O React possui uma arquitetura onde seus componentes são organizados hierarquicamente
- Esse compartilhamento é feito através de herança
- Não é possível que componentes que estejam em um mesmo nível acessem diretamente dados uns dos outros



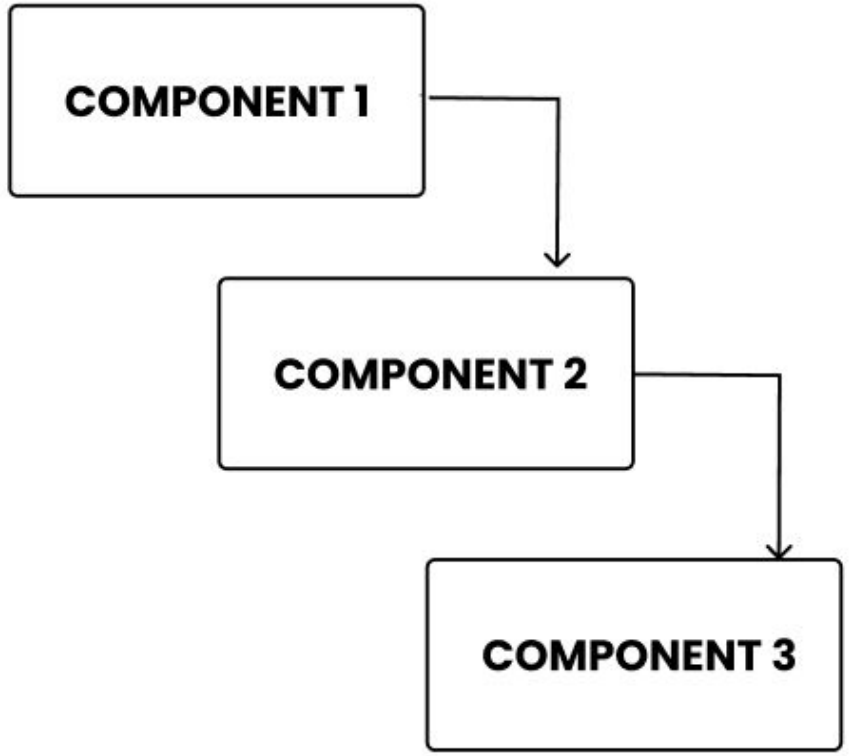
Estado Local vs. Estado Global

- Além disso, um componente pai também não tem acesso direto a dados dos filhos
- O problema de passar props por vários níveis da árvore é conhecido como Prod Drilling



Prop Drilling

- Passagem de props por vários níveis da árvore
- Componentes intermediários apenas repassam dados
- Por que é um problema:
  - Código difícil de manter e refatorar
  - "Poluição" dos componentes intermediários
  - Rastrear o fluxo de dados fica complexo



Prop Drilling

EXEMPLO

- Mostrar um componente App com um estado usuario
- App passa usuario para PaginaPrincipal
- PaginaPrincipal (que não usa usuario) passa para BarraDeNavegacao
- BarraDeNavegacao (que também não usa) passa para AvatarUsuario
- AvatarUsuario finalmente usa a propriedade usuario

Exemplo prático

- Entrar no site da W3Schools

https://www.w3schools.com/react/react_usecontext.asp

- Realizar o primeiro exemplo, na seção: The Problem

Arquitetura Flux

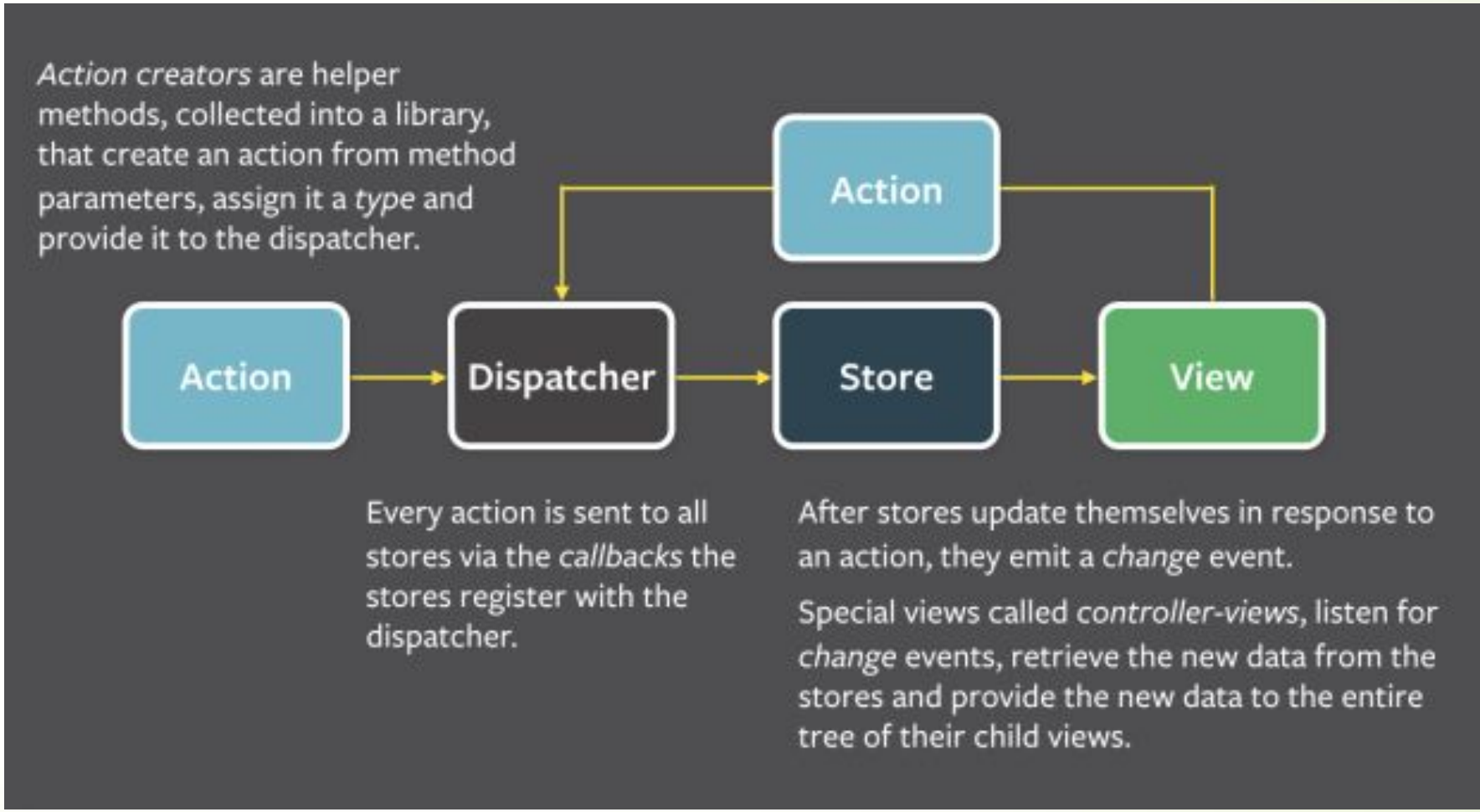
SOLUÇÃO CONCEITUAL

- Uma arquitetura, não uma biblioteca, criada pelo Facebook
- Propõe um fluxo de dados estritamente unidirecional
- Objetivo: Tornar o fluxo de estado mais previsível e fácil de depurar
- Mantra: "Dados fluem em uma única direção."

Arquitetura Flux

OS 4 PILARES DO FLUX

- Action
- DIspatcher
- Store
- View



Arquitetura Flux

OS 4 PILARES DO FLUX

1. Actions (Ações):

- Objetos que descrevem o que aconteceu
- Contêm um type e, opcionalmente, um payload (dados)
- Ex: { type: 'ADD_ITEM_TO_CART', payload: { itemId: 123 } }

Arquitetura Flux

OS 4 PILARES DO FLUX

2. Dispatcher (Despachante):

- Um ponto central que recebe as Actions e as envia para todas as Stores registradas
- Garante que não haja ações simultâneas

Arquitetura Flux

OS 4 PILARES DO FLUX

3. Stores (Armazenamentos):

- Contêm o estado da aplicação e a lógica de como ele deve mudar em resposta a uma Action

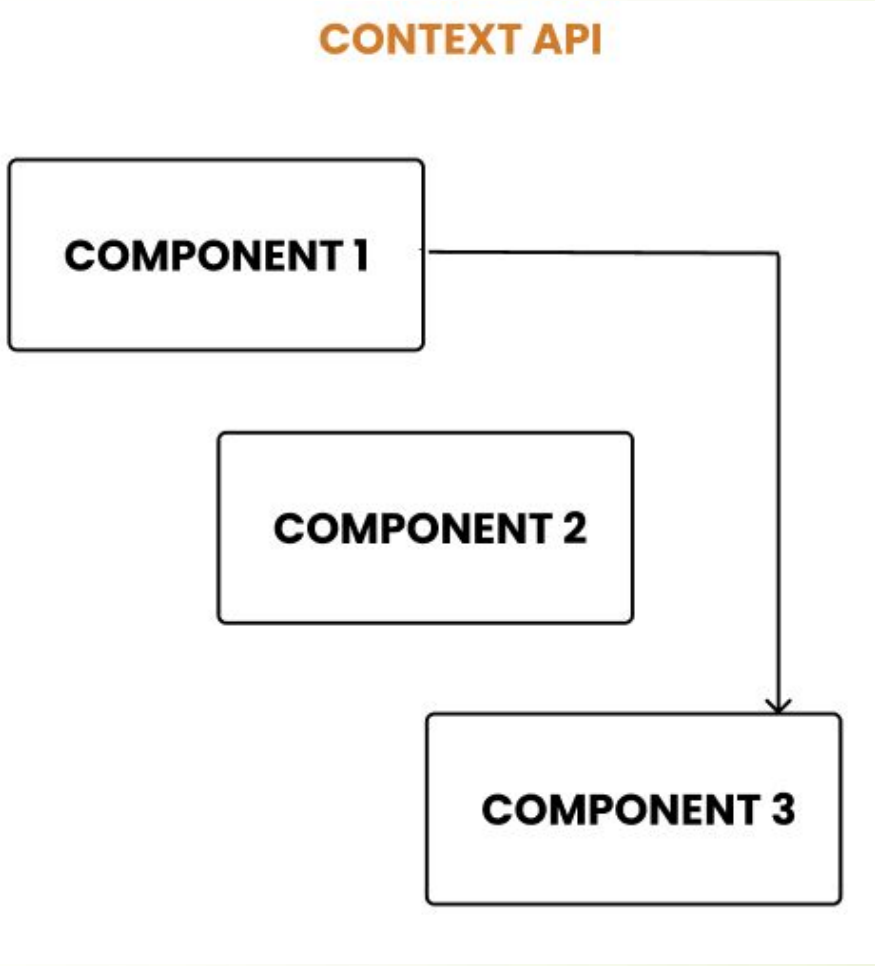
4. Views (Visões):

- Os componentes (React) que exibem os dados das Stores e disparam Actions com base na interação do usuário

Context API

SOLUÇÃO NATIVA DO REACT

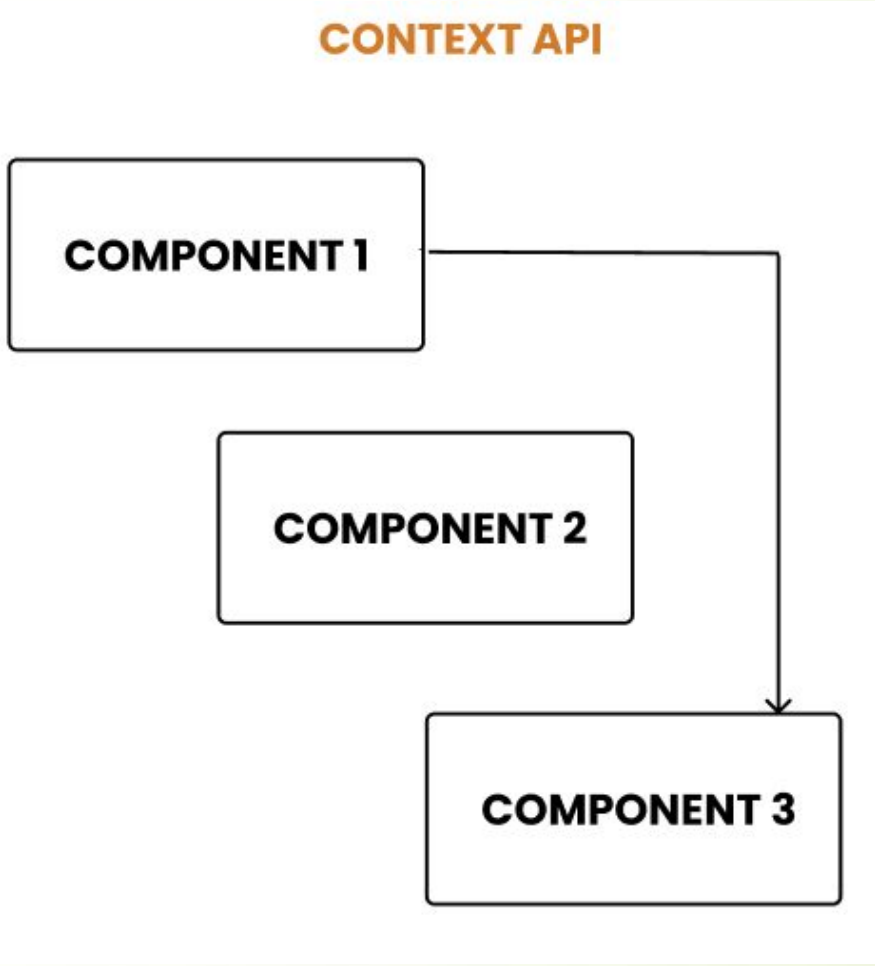
- O React oferece uma solução eficaz para quebrar essa hierarquia e permitir a existência de states globais
- Possibilita compartilhar states diretamente entre dois componentes
- Mesmo que eles não façam parte de uma mesma hierarquia



Context API

QUANDO USAR

- Para dados que não mudam com frequência
- Para evitar "prop drilling" em cenários mais simples
- Exemplos: Tema da UI, informações de autenticação, idioma



Context API

COMO USAR

Passo 1: Criar o Contexto:

```
const MeuContexto = React.createContext(valorPadrao);
```

- Cria um objeto de contexto
- O valorPadrao é usado quando um componente não tem um Provider acima dele na árvore

Context API

COMO USAR

Passo 2: Prover o Contexto

```
<MeuContexto.Provider value={/* algum valor */}>
```

- Este componente "abraça" a parte da sua árvore que precisa do acesso ao estado
- Todos os filhos poderão consumir o value

Context API

COMO USAR

Passo 3: Consumir o Contexto

```
const valor = useContext(MeuContexto);
```

- O hook useContext permite que um componente funcional leia e se inscreva nas mudanças do contexto

Exemplo prático

- Entrar no site da W3Schools, na url abaixo:

  https://www.w3schools.com/react/react_usecontext.asp

- Realizar os exemplos da seção: The Solution

Exemplo prático 2

- Partir do exemplo realizado do W3Schools
- Resolver esse exemplo utilizando componentizaçao com arquivos externos
- Criar, por exemplo um componente: Component1.jsx
- Utilizar esse componente no App.jsx mantendo o conceito de useContext

Introdução ao Redux

- Quando o Context API não basta?
- Em estados que mudam com alta frequência ou lógica de negócio complexa
- Redux: Um Contêiner Previsível de Estado
- É uma biblioteca de gerenciamento de estado inspirada no padrão Flux

Introdução ao Redux

PRINCÍPIOS FUNDAMENTAIS

- Fonte única de verdade: O estado de toda a aplicação é armazenado em uma única árvore de objetos dentro de uma única **Store**
- O estado é somente leitura: A única maneira de mudar o estado é emitindo uma **Action**, um objeto que descreve o que aconteceu
- As mudanças são feitas com funções puras: Para especificar como a árvore de estado é transformada por uma Action, escrevemos **Reducers** puros

Introdução ao Redux

O FLUXO DO REDUX

- Action: Objeto com type e payload
- Reducer: Função que recebe (state, action) e retorna um novo estado
- Store: Objeto que mantém o estado da aplicação
- Dispatch: Método para disparar a ação

Exemplo Prático

- Acessar o site do React Redux:

  https://react-redux.js.org/introduction/getting-started

- Entender como instalar a biblioteca

- Entender de maneira geral o uso da biblioteca em React

Melhores Práticas

QUANDO USAR GLOBAL VS. LOCAL

- Local (useState): Formulários, estados de UI (dropdown aberto), carregamento local
- Global (Context/Redux): Autenticação, dados do carrinho de compras, preferências do sistema
- **Dica**: Não coloque tudo no estado global para não prejudicar a performance (re-renders desnecessários)

Melhores Práticas

- Lembrete sobre o Flux:
  - Um padrão de arquitetura, não uma ferramenta
  - Foca no fluxo de dados unidirecional
- Context API:
  - Ideal para: Dados de baixa frequência (tema, usuário, etc.)
  - Prática: Mantenha contextos focados em um único propósito para evitar re-renderizações desnecessárias

Melhores Práticas

- Redux:
  - Ideal para: Estados complexos e de alta frequência (formulários, carrinhos, dados de servidor)
  - Prática: Use o Redux Toolkit para simplificar a configuração, reduzir o boilerplate e seguir as melhores práticas por padrão

Melhores Práticas

- Regra de Ouro:
  - Comece com o estado local (useState)
  - Se precisar passar props por mais de 2-3 níveis, considere a Context API
  - Se a lógica de estado se tornar complexa ou as atualizações forem muito frequentes, migre para o Redux

Exercícios

- Objetivo: Implementar um seletor de tema (claro/escuro) usando a Context API
- Tarefa:
  - Crie um ThemeContext para armazenar o tema atual ('light' ou 'dark') e uma função para alterá-lo
  - Use o ThemeProvider para envolver a aplicação
  - Crie um componente BotaoSeletor que usa o useContext para ler e alterar o tema
  - Aplique estilos diferentes a um componente Pagina com base no tema consumido do contexto

Exercícios

- Objetivo: Gerenciar a lista de itens de um carrinho de compras usando Redux.
- Tarefa:
  - Configure uma Store do Redux
  - Crie um cartReducer que lide com as ações ADD_ITEM e REMOVE_ITEM
  - Crie componentes ListaDeProdutos (para disparar ADD_ITEM) e Carrinho (para exibir os itens e disparar REMOVE_ITEM)
  - Conecte os componentes à Store do Redux para ler o estado e despachar ações

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Gerenciamento de Estado Global**
  - Criação de um contexto global para tarefas/projetos
  - Criar um componente StatusHeader que mostra "Você tem X tarefas pendentes" consumindo esse contexto global.
  - Mover o estado das tasks para um TaskContext (opcional, mas recomendado para aprender a compartilhar dados entre componentes distantes).
  - Criar um ThemeContext para alternar entre modo Claro e Escuro.
- Funcionalidade: Criar um tema (Dark/Light Mode), mover as tarefas para um Contexto Global para evitar "Prop Drilling"
- Entrega: Um botão de "sol/lua" que troca as cores de todo o app instantaneamente

Referências

- TerraLAB. Gerenciamento de estados globais no React – Como utilizar Context API. DIsponível em: https://www2.decom.ufop.br/terralab/gerenciamento-de-estados-globais-no-react-como-utilizar-context-api/

- W3Schools. React useContext Hook. Disponível em: https://www.w3schools.com/react/react_usecontext.asp

- React Redux. Getting Started with React Redux. Disponível em: https://react-redux.js.org/introduction/getting-started